

PROJECT ATLAS

ATLAS-DES-001: SYSTEM & SOFTWARE ARCHITECTURE

1. Physical Hardware Topology

The system is a self-contained, battery-powered real-time display unit. Power is managed via high-capacity Lithium cells and high-current DC-DC conversion.

Category	Component	Connection Interface
Compute	Intel NUC PC (Windows 11)	Primary System Controller
Operator Display	Small Low-Power LCD	HDMI #1 / USB Power
Public Display	LED Matrix (P3/P10)	HDMI #2 -> HUB75 Tx -> HUB75 Rx
Interface Kit	HDMI to LED Video Interface	HDMI Data / HUB75 Ribbon
Power Source	200Ah Lithium Battery	12V Bus
Voltage Reg	DCDC 12V/18V & 5V (20A)	Hardwired terminal block

2. Software Conceptual Design (Atlas Engine)

The software follows a **Modular Observability** pattern. Components do not call each other directly; they react to changes in the central State Machine.

Core Module: AtlasStateEngine

The "Marrow" of the project. It tracks the system lifecycle: Startup, Ready, Armed, Running, Finished, Cancelled.

2.1 Functional Modules

- **AtlasUI:** Handles user input, validation, and branding. It sends "Requests" to the Engine.

- **AtlasRenderer:** Takes the current State_Active and paints it to the borderless HDMI canvas on Display #2.
- **AtlasTimerEngine:** A high-precision thread specifically for decrementing windows.
- **AtlasPersistence:** Handles SQLite interactions for race logging and config storage.

3. Data Transfer Model

Atlas uses **Immutable State Snapshots** for the Active Heat. Once a heat is activated, the data is copied and locked. Adjusting "Next Heat" in the UI does not influence the "Active Heat" until the next logic transition.

4. Traceability Alignment

This design fulfills: **URS-001** (NUC Hosting), **URS-003** (Dual Display), **URS-004** (LED HDMI Output), and **URS-040** (State-based Activation).